

DATA 581

Modeling and Simulation II



Markov Processes and a Brief Look at Financial Time Series

Outline

1. Random Walk
2. Brownian Motion
3. The ARCH Model

These slides are brought to you by R and

```
set.seed(360926)
```

Random Walk

Suppose $\{B_n\}$ is a sequence of independent Bernoulli random variables with $p = 0.5$. i.e. $P(B_n = 1) = 1 - P(B_n = 0) = 0.5$.

Let $Z_0 = 0$ and

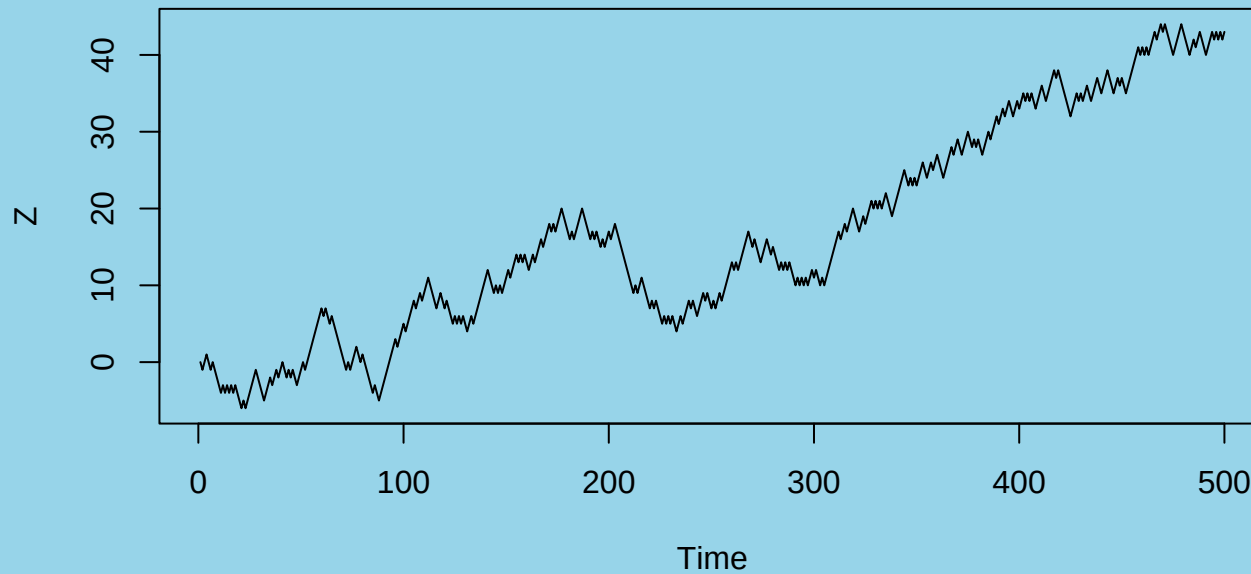
$$Z_n = Z_{n-1} + 2(B_n - 1/2), \text{ for } n = 1, 2, \dots$$

Simulation:

```
N <- 500 # number of time steps
Z <- numeric(N)
for (i in 2:N) {
  Z[i] <- Z[i-1] + 2*(rbinom(1, 1, 0.5) - 0.5)
}
```

Random Walk

`ts.plot (Z)`



This Markov process is often used as a first approximation to stock market data.

Random Walk to Brownian Motion

Now, let Δ be a positive number (that we will eventually let tend to 0).

Z_t defined below is another random walk

$$Z_t = Z_{t-\Delta} + 2\sqrt{\Delta}(B_t - 1/2).$$

Note that $E[Z_t - Z_{t-\Delta}] = 0$ for all Δ .

$\text{Var}(Z_t - Z_{t-\Delta}) = \Delta$ for all Δ . Therefore, $\text{Var}(Z_t) = t$.

Suppose that (s_1, t_1) and (s_2, t_2) do not intersect. Then $(Z_{t_1} - Z_{s_1})$ and $(Z_{t_2} - Z_{s_2})$ are independent. This is an *independent increments* property.

Z_t is a Markov process, and the limiting version of Z_t as $\Delta \rightarrow 0$ is called *Brownian Motion*. It is the simplest example of a *Gaussian* process.

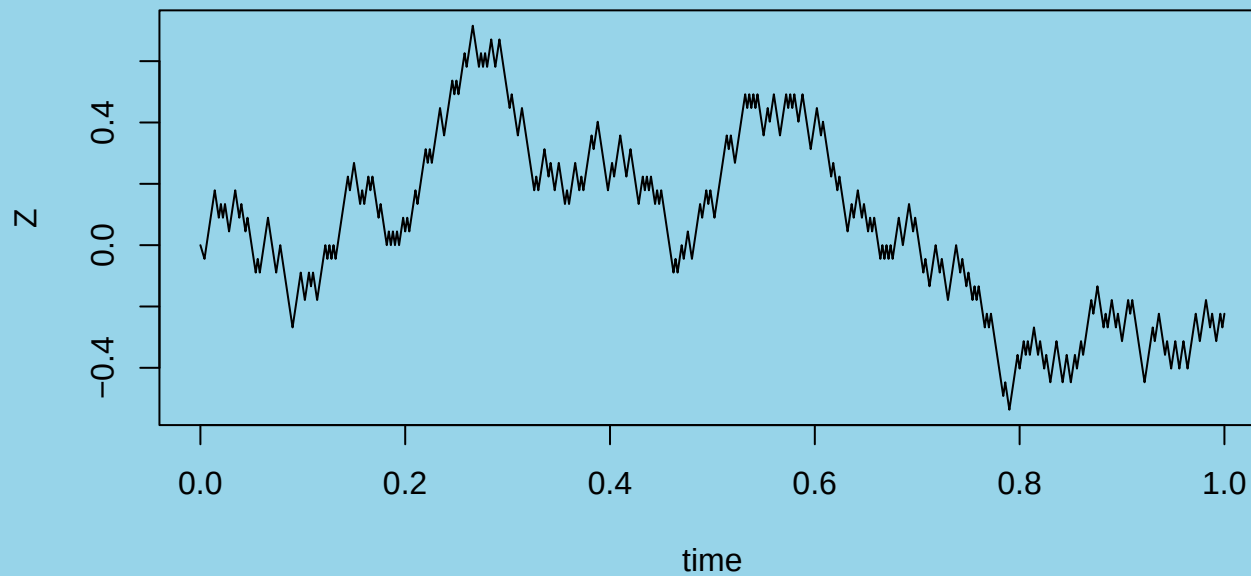
Brownian Motion

Simulation:

```
N <- 500 # number of time steps
Delta <- 1/N
time <- numeric(N)
Z <- numeric(N)
for (i in 2:N) {
  Z[i] <- Z[i-1] +
    2*sqrt(Delta)*(rbinom(1, 1, 0.5) - 0.5)
  time[i] <- i*Delta
}
```

Brownian Motion

```
plot(Z ~ time, type="l")
```



Brownian Motion

Simulation to show that Z_t has a normal distribution with mean 0 and variance t :

```

Nsims <- 500
Zt <- numeric(500)
for (j in 1:Nsims) {
  N <- 500 # number of time steps
  Delta <- 1/N
  time <- numeric(N)
  Z <- numeric(N)
  for (i in 2:N) {
    Z[i] <- Z[i-1] +
      2*sqrt(Delta)*(rbinom(1, 1, 0.5) - 0.5)
    time[i] <- i*Delta
  }
  Zt[j] <- Z[N] # Z[N] contains the current Zt value
}

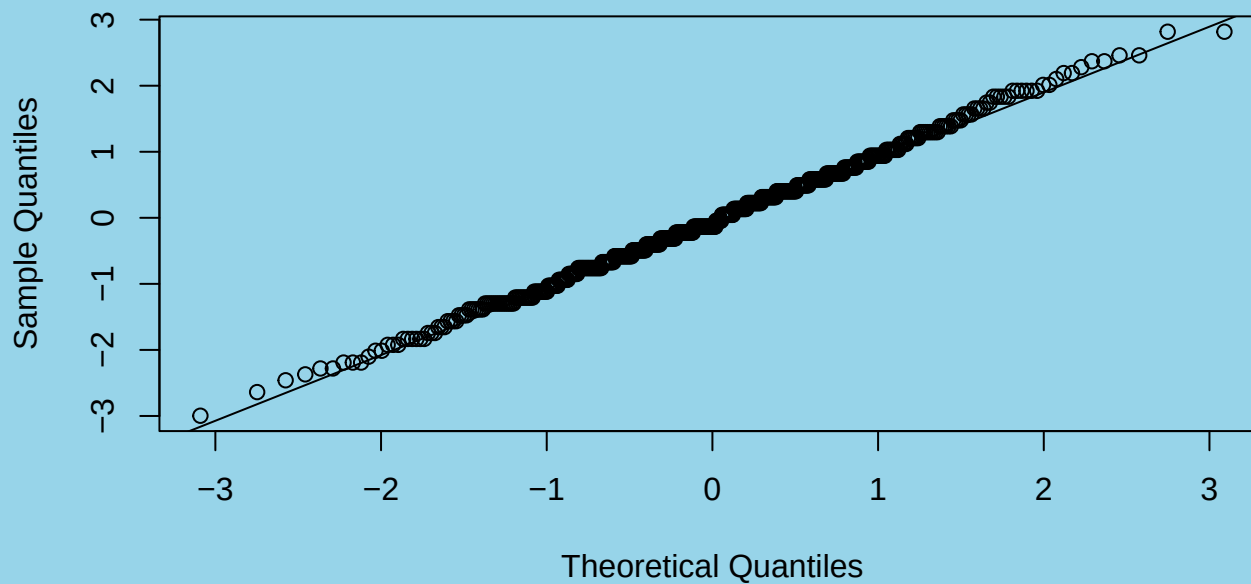
```


Brownian Motion

```
qqnorm(Zt)
```

```
qqline(Zt)
```

Normal Q-Q Plot



According to the normal QQ-plot, Z_t appears to be normally distributed.

Geometric Brownian Motion and Financial Time Series

Because of the way that Brownian motion can trend in any direction, it is possible for it to yield negative values at times.

For many financial time series, including sequences of stock prices, and stock indices, negative values are impossible.

By exponentiating the Brownian motion, positive values are guaranteed.

Since Brownian motion values are really normally distributed, these exponentiated values must be lognormally distributed.

The resulting process is often called geometric Brownian motion, and is sometimes called a lognormal diffusion, especially in the case where the Brownian motion has been replaced with a more general Gaussian process.

Geometric Brownian Motion and Financial Time Series

A drift term is usually included in the geometric Brownian motion model as well.

The geometric Brownian motion process $G(t)$ is given by

$$G(t) = G(0)e^{\mu t + \sigma B(t)}$$

where μ is the drift parameter, $B(t)$ is the Brownian motion and σ is called the diffusion parameter. It is just the standard deviation of the cumulating normals.

Taking the log of $G(t)$ and gives

$$\log(G(0)) + \mu t + \sigma B(t).$$

As a model for observed data, we could think of differencing the data once to obtain normally distributed values with mean μ and variance σ^2 .

Geometric Brownian Motion and Financial Time Series

Example. European Stock Market Index (German Exchange).

```
difflogIbis <- diff(log(EuStockMarkets[,1])) # log and diff
mu <- mean(difflogIbis) # estimate mean
sigma <- sd(difflogIbis) # estimate sd
```

The drift and diffusion parameters are estimated as

```
mu; sigma

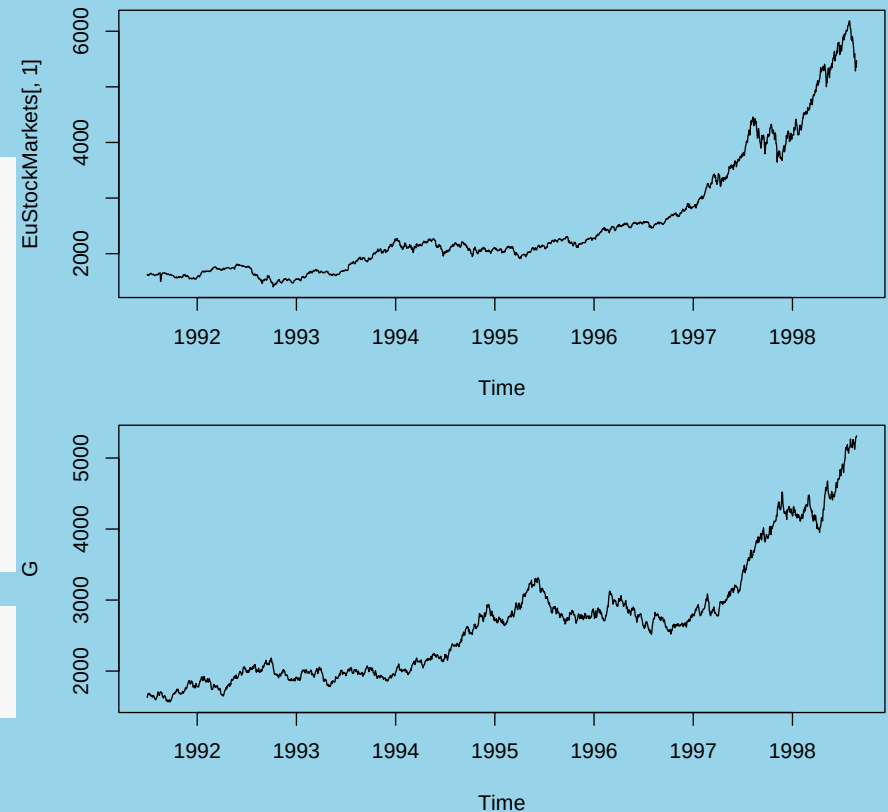
## [1] 0.0006520417
## [1] 0.01030084
```

Geometric Brownian Motion and Financial Time Series

Example. Simulate stock index-like data.

```
n <- nrow(EuStockMarkets)
B <- arima.sim(n-1, model=
  list(order=c(0,1,0)))
X <- sigma*B + mu*(1:n)
  # t is discrete time here
G <- EuStockMarkets[1,1]*exp(X)
G <- ts(G, start = c(1991, 130),
  frequency = 260)
```

```
ts.plot(EuStockMarkets[,1])
ts.plot(G)
```



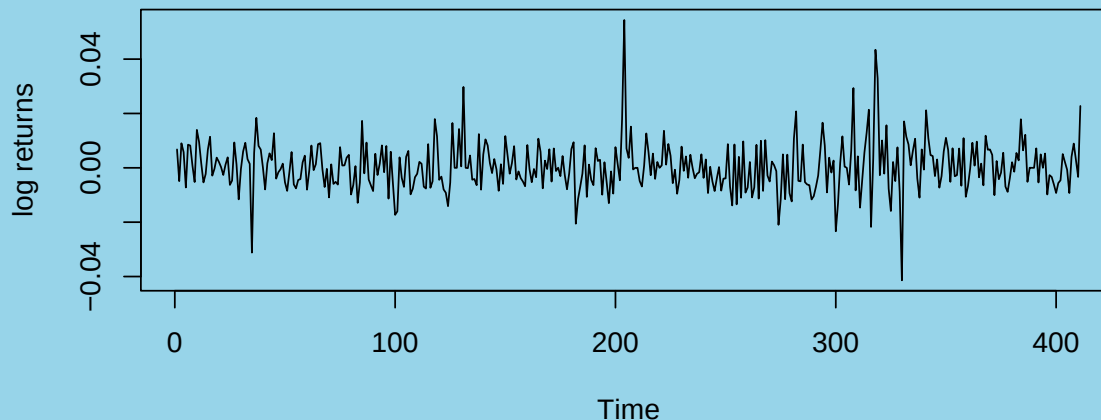
Other simulated series will look different, of course. These kinds of simulations can be used to estimate prices of various kinds of financial options. Note that the basic building block of the simulation of this kind of calculation is ability to simulate normal, and hence, lognormal data.

Allowing the Process Variance to Change: the ARCH model

The ARCH model, and its more sophisticated variant, GARCH, are important models in finance.

The following trace plot shows daily log returns for the FTSE (Financial Times Stock Exchange) for 1991 and 1992:

```
logreturns <- diff(log(EuStockMarkets[1:412, 4]))
ts.plot(logreturns, ylab="log returns")
```



The log return for day t is the logarithm of x_t/x_{t-1} . It gives an indication as to how well the market is doing. A positive log return means that the market went up.

ACF of the Log Returns

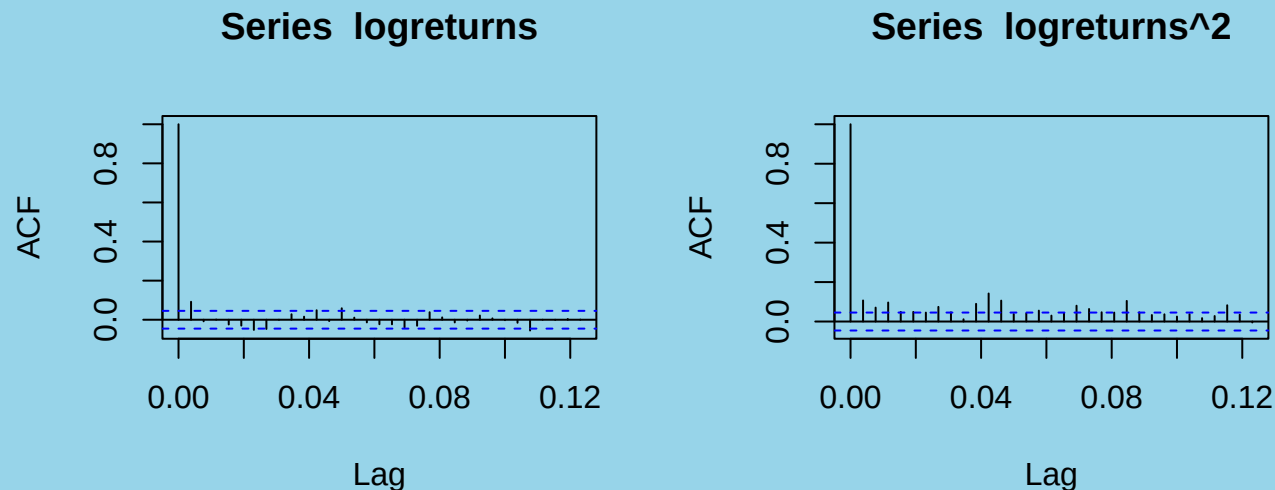
Analyze the full data set:

```
logreturns <- diff(log(EuStockMarkets[, 4]))
```

```
par(mfrow=c(1, 2))
```

```
acf(logreturns)
```

```
acf(logreturns^2)
```



There is a slight autocorrelation at lag 1 in the raw log returns, but there is somewhat more autocorrelation in the squared log returns (right panel).

Modelling Log Returns with ARCH

A model which gives similar behaviour to the daily log returns for the FTSE is the following:

For day t , the log return is given by

$$y_t = s_t Z_t$$

where

$$s_t = \sqrt{a_0 + a_1 y_{t-2}^2 + a_2 y_{t-1}^2}$$

and Z_t is a standard normal random variable.

The parameter values are $a_0 = 0.00001$, $a_1 = 0.1$ and $a_2 = 1.0$.

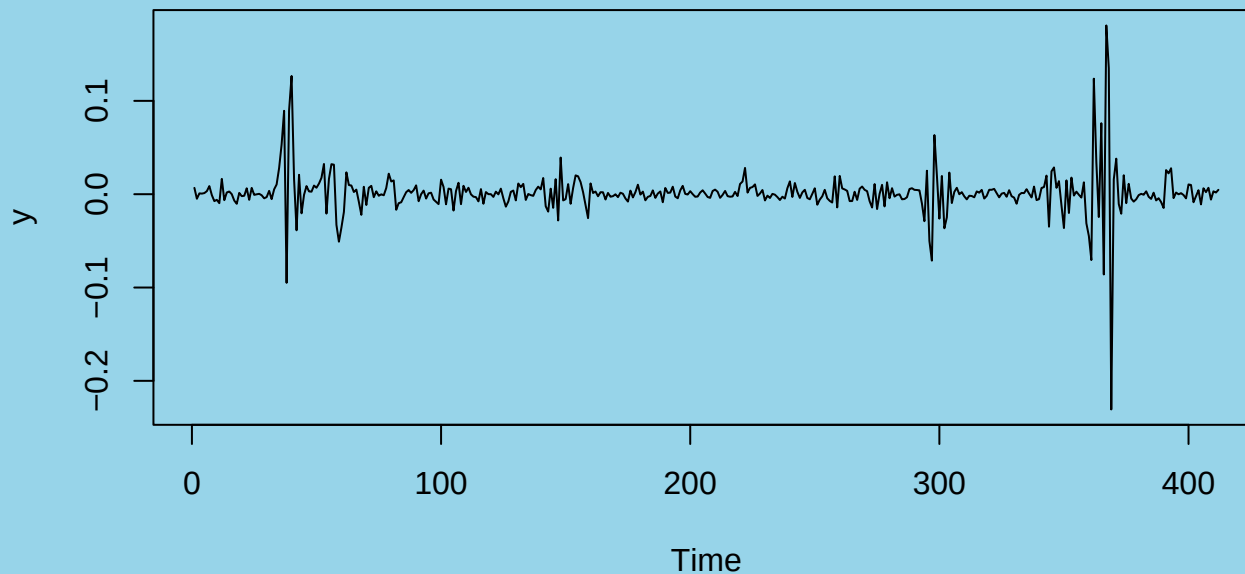
We can start of a simulation by setting the first two values of y to the first two values from the observed log returns.

An ARCH Model Example

```
n <- 412 # number of days for the simulation
a <- c(1, .1); a0 <- .00001
y <- numeric(n); y[1:2] <- logreturns[1:2]
for (t in 3:n){
  s <- sqrt(a0 + a[1]*y[t-1]^2 + a[2]*y[t-2]^2)
  y[t] <- s*rnorm(1)
}
```

An ARCH Model Example

```
ts.plot(y)
```



As in the actual series, the simulated values hover between ± 0.01 but occasionally between ± 0.04 . We might use this model to predict future behaviour of the FTSE, such as how long it might take to exceed some value, such as 0.05 .

An ARCH Model Example

Suppose we want to simulate the ARCH process until the first time it exceeds 0.05.

We don't know beforehand when this will occur, we wouldn't know how to stop the `for()` loop at the right time, so we should use a `while()` loop.

An ARCH Model Example

```
ARCH <- 0; t <- 2
y[1:2] <- logreturns[1:2]
while (y[t] < .05){
  t <- t+1
  s <- sqrt(a0 + a[1]*y[t-1]^2 + a[2]*y[t-2]^2)
  y[t] <- s*rnorm(1)
}
print(t)

## [1] 234
```

By repeatedly running this simulation, we could obtain a distribution of the times until we would expect the log returns to first exceed .05. This kind of information would be useful, for example, in pricing certain financial options.

Summary

- 1. Continuous time-continuous state Markov processes provide another important class of models. Brownian motion arises as a certain limit of the random walk.**
- 2. Because these processes can be approximated by discrete time counterparts, it is possible to simulate approximate paths of these processes using time series simulators.**
- 3. Financial time series are often modelled with geometric Brownian motion, which is effectively a lognormal process. Data with changes in variance (called *volatility*) can be modelled with ARCH (or GARCH) processes which are appropriate when there is little observed autocorrelation but there is still dependency in the data.**